

EPBL PROJECT REPORT

Code Companion

Student Details:

- | | |
|---------------------|---------------------|
| 1. A Koushiksai | Roll No: 23881A04C7 |
| 2. Tatikonda Sravya | Roll No: 23881A04C1 |
| 3. Miryala Varshini | Roll No: 23881A04C5 |

Domain: Full-Stack Web Development

Technologies: React.js, Node.js, Express.js,
Google Gemini API

1 Abstract

Code Companion is an AI-powered full-stack web application developed to assist programmers in understanding, analyzing, and improving source code. The platform integrates a modern React-based frontend with a secure Node.js and Express.js backend, utilizing Google's Gemini API to perform intelligent code analysis. Users can sign in to a personal workspace, submit source code through an interactive Monaco Editor, and receive detailed explanations, estimated time and space complexity, optimization suggestions, and refactored implementations presented as a side-by-side diff.

Beyond the core analysis engine, the application now includes user authentication, a persistent Optimization Dashboard that archives past analyses for later retrieval, and a PDF export feature for sharing reports. The application demonstrates practical implementation of REST APIs, frontend-backend communication, AI integration, session-based authentication, and modern web development practices. By providing instant feedback and code optimization recommendations, Code Companion simplifies the software development workflow while helping programmers write cleaner, more efficient, and maintainable code.

2 Introduction

Writing efficient and maintainable software requires developers to understand program logic, identify performance bottlenecks, and optimize algorithms. Manual code review is often time-consuming, especially for beginners who may struggle to analyze algorithmic complexity and detect potential improvements.

Recent advancements in Large Language Models (LLMs) have enabled intelligent systems capable of understanding source code and generating meaningful explanations. Integrating these capabilities into a developer-friendly environment can significantly improve productivity and learning.

Code Companion is developed under the domain of Full-Stack Web Development and Artificial Intelligence. The application combines React.js, Express.js, and the Google Gemini API to create an interactive platform where developers can securely sign in, analyze code, estimate computational complexity, identify possible improvements, view an optimized rewrite side by side with the original, and revisit prior analyses through a personal dashboard.

3 System Architecture

Code Companion is built using a classic three-tier architecture comprising the Presentation Layer (Frontend), Application Layer (Backend), and Integration Layer (AI Services). This decoupled design ensures separation of concerns, scalability, and ease of maintenance.

3.1 Frontend Architecture

The frontend is constructed using React.js, employing a component-based architecture. Key UI modules include:

- **Authentication Screens:** A dedicated Sign In / Sign Up flow gates access to the workspace, ensuring each user's analyses and history remain private to their account.
- **Code Workspace (Analyzer):** Utilizes the Monaco Editor (the core engine behind VS Code) to provide syntax highlighting, line numbering, drag-and-drop file uploads, and a native IDE-like user experience across multiple languages (C++, Java, Python).
- **Optimization Telemetry Panel:** Renders structured analysis data, splitting the output into dedicated sections for time/space complexity metrics ($O(n)$ notation), the isolated bottleneck explanation, and a side-by-side diff of the original versus optimized code.
- **Dashboard Panel:** A separate tab that lists the user's Optimization Archives, showing total optimizations performed, the timestamp of the last activity, the active analysis engine, and quick links to reload any past submission back into the workspace.

3.2 Backend Architecture

The backend service is powered by Node.js and Express.js, functioning as a middleware proxy between the client-side UI and external AI APIs. It manages:

- **Authentication & Session Management:** Handles user sign-up and login, and scopes all stored analyses to the authenticated user.
- **API Routing:** Exposes REST endpoints (e.g., POST `/api/analyze`) to handle code submission payloads securely.

- **AI API Integration:** Sanitizes the incoming user code, injects strict system instructions for structured output formatting, and interacts with the Google Gemini API using secure environment variables.
- **Optimization History Storage:** Persists each analysis result (language, complexity metrics, bottleneck summary, and diff) so it can be retrieved later from the Dashboard.
- **Report Export:** Generates a downloadable PDF summary of an analysis for offline sharing and record-keeping.

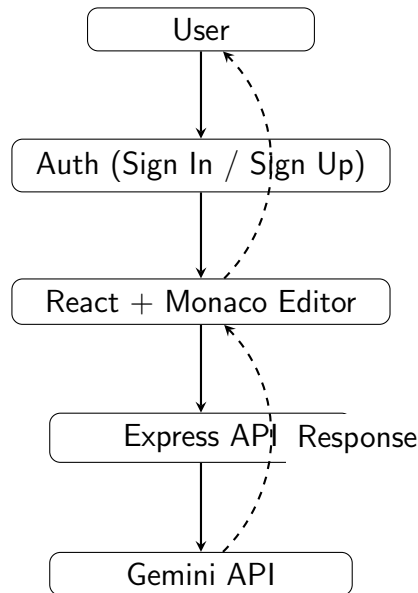


Figure 1: System Architecture Diagram of Code Companion

4 Technology Stack

5 Implementation

The backend of Code Companion is developed using Express.js. The API receives source code from the React frontend, forwards it to the Google Gemini API, and returns structured analysis to the client. Each result is also persisted so it can be listed later in the Optimization Dashboard.

Layer / Component	Core Technologies Used
Frontend	React.js, Tailwind CSS, Monaco Editor
Backend	Node.js, Express.js
Authentication	Session/Token-based user accounts
AI Model	Google Gemini API (Gemini 3.1 Flash-Lite)
Report Export	PDF generation service
Deployment	Vercel, Render
HTTP Client	Axios

Table 1: Technology Stack Used

5.1 Express.js API Route

Listing 1: Express.js API Route

```

1 app.post("/api/analyze", async (req, res) => {
2     const { code, language } = req.body;
3     const userId = req.user.id; // set by auth
      middleware
4
5     const model = genAI.getGenerativeModel({
6         model: "gemini-3.1-flash-lite"
7     });
8
9     const prompt = `Analyze this ${language} code`;
10
11     const result = await model.generateContent([
12         prompt,
13         code
14     ]);
15
16     const analysis = JSON.parse(result.response.text());
17
18     // Persist to the user's optimization history for
      the Dashboard
19     await OptimizationHistory.create({
20         userId,
21         language,
22         ...analysis
23     });
24
25     res.json(analysis);
26 });

```

6 Results and Discussion

The developed application was successfully tested using multiple source code examples written in C++, Java, and Python. The system accurately generated explanations of program logic, estimated time and space complexity, identified potential improvements, and produced optimized versions of the submitted code.

The communication between the React frontend, Express.js backend, and Google Gemini API functioned reliably, providing analysis within a few seconds for most code snippets. The application offers a clean and intuitive interface through the Monaco Editor, allowing users to write, paste, or drag-and-drop source code and receive detailed feedback instantly.

6.1 Authentication

Access to the workspace is gated behind a Sign In / Sign Up screen. This ensures that each user's submitted code and optimization history remain private and are only visible after successful authentication.

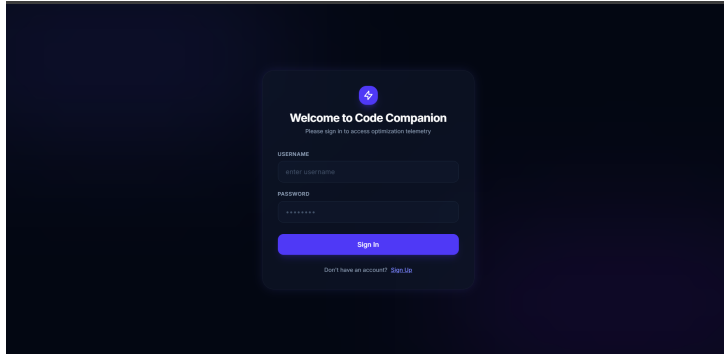


Figure 2: Sign In Screen of Code Companion

6.2 Application Interface

Figure 3 illustrates the main Analyzer interface of Code Companion. The application provides a professional development environment featuring the Monaco Editor, language selection (C++, Java, Python), drag-and-drop file support, and a dedicated Optimization Telemetry panel awaiting input.

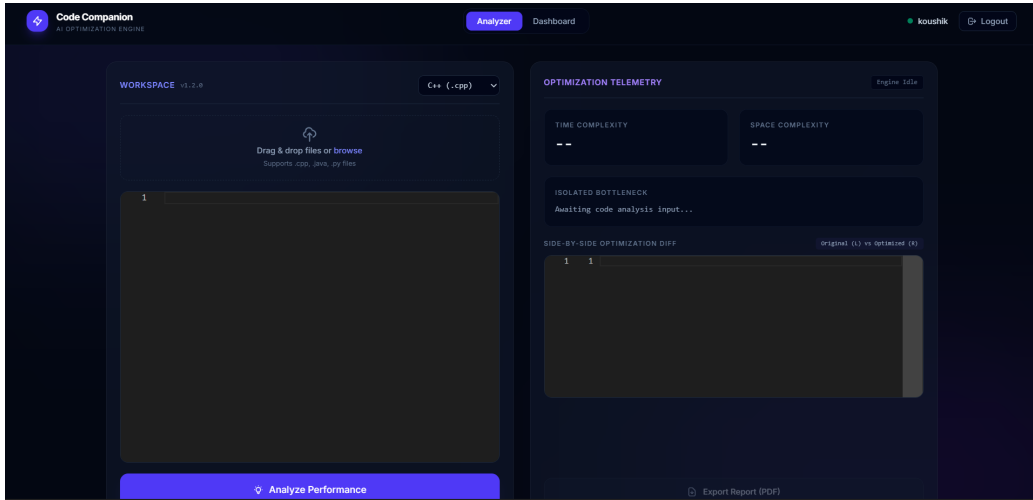


Figure 3: Analyzer Workspace of Code Companion (Idle State)

6.3 Code Analysis and Side-by-Side Diff

After submitting source code, the backend forwards the request to the Google Gemini API, which generates a structured response. The application displays the estimated time and space complexity, an isolated bottleneck explanation, and a side-by-side diff view contrasting the original code (left) with the AI-optimized version (right), with removed and added lines highlighted for quick review.

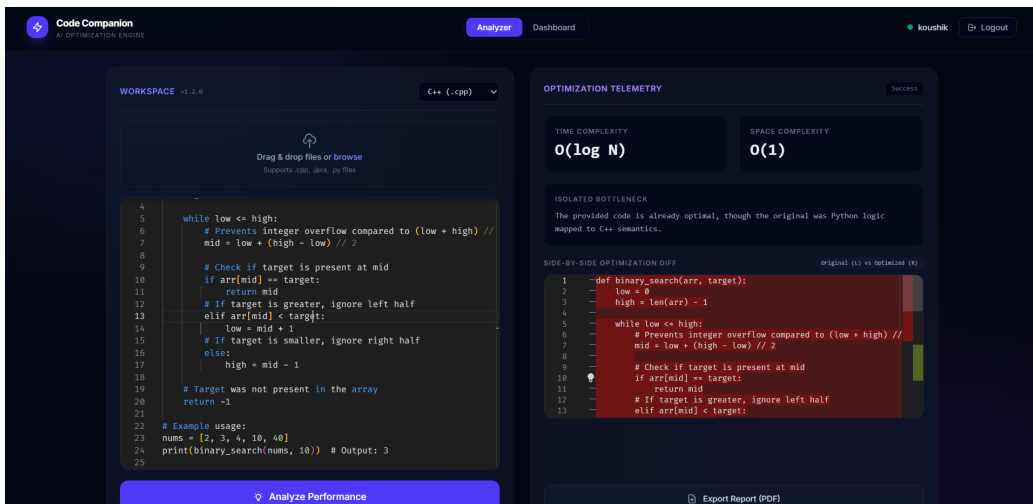


Figure 4: AI-Generated Code Analysis with Side-by-Side Optimization Diff

6.4 Optimization Dashboard

A dedicated Dashboard tab gives users an overview of their activity: the total number of optimizations performed, the timestamp of their last activity, and the active analysis engine. Below this summary, an Optimization Archive lists each past submission with its language, complexity results, and bottleneck summary, and allows the user to reload any entry back into the Analyzer workspace for further review.

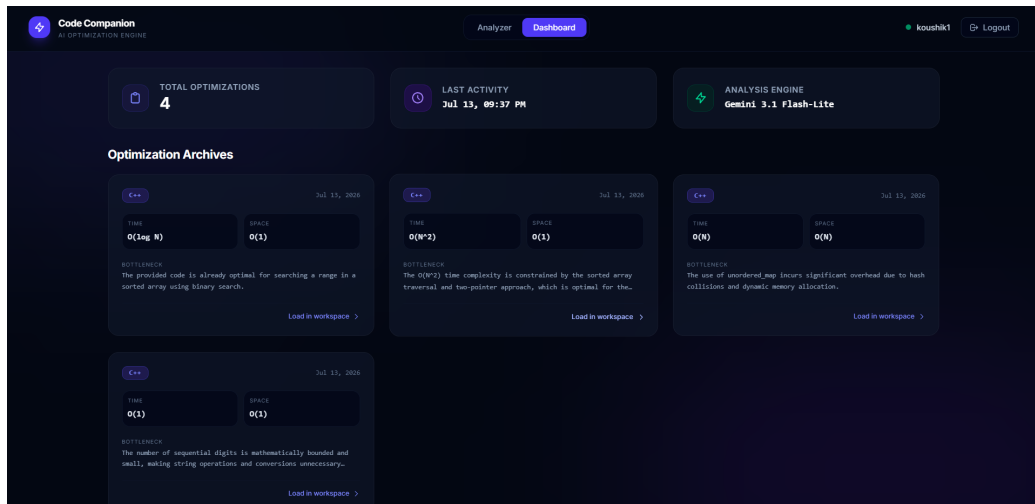


Figure 5: Optimization Dashboard Showing Archived Analyses

The experimental results demonstrate that Code Companion successfully integrates modern web technologies with generative AI to provide an effective, secure, and persistent code analysis platform. The application simplifies the process of understanding algorithms, estimating computational complexity, and improving source code quality, making it a valuable learning and productivity tool for students and software developers.

7 Features and Functionalities

The completed application provides a comprehensive suite of tools designed to accelerate a developer's workflow:

1. **User Authentication:** Secure Sign In / Sign Up flow so that each user's code submissions and history stay private to their account.
2. **Interactive Code Entry:** Support for multiple popular programming

languages, including C++, Java, and Python, using the Monaco Editor environment with drag-and-drop file uploads.

3. **Algorithmic Complexity Estimation:** Automatically computes Big O notation for both time and space parameters, helping users identify potential performance blocks before deployment.
4. **Isolated Bottleneck Detection:** Highlights the specific line or construct responsible for the dominant cost in the submitted code.
5. **Side-by-Side Optimization Diff:** Presents the original and AI-optimized code next to each other with inline highlighting, making it easy to see exactly what changed.
6. **Contextual Refactoring:** Provides completely optimized, syntactically valid versions of the inputted code, focusing on edge-case handling and modern syntax standards.
7. **Optimization Dashboard:** Archives every past analysis, letting users track total optimizations performed and reload any previous submission into the workspace.
8. **PDF Report Export:** Allows users to export a given analysis, including the diff and complexity metrics, as a shareable PDF report.

8 Future Work

While functional as a full-featured web platform, future iterations of Code Companion aim to include:

- **IDE Extensions:** Porting the backend service into dedicated VS Code and JetBrains marketplace extensions for native inline access.
- **Multi-File Analysis:** Expanding the prompt parsing capabilities to allow entire workspace repositories to be uploaded and analyzed contextually.
- **Local LLM Support:** Introducing configuration options for developers to use local, open-source models (e.g., Ollama, Llama 3) for enterprise codebases requiring strict offline privacy.
- **Team Workspaces:** Extending the authentication system to support shared, organization-level dashboards so teams can review each other's optimization history.

9 Conclusion

Code Companion successfully demonstrates how modern AI models can be tightly coupled with full-stack web architectures to deliver tangible utility to developers. By combining React’s highly responsive UI with Node.js and the Gemini API, and layering in user authentication, a persistent optimization dashboard, and PDF export, the platform delivers fast, contextual, and highly educational insights into source code, backed by a secure and personalized user experience.